



PDF Download
3725783.3764393.pdf
19 March 2026
Total Citations: 0
Total Downloads: 497

Latest updates: <https://dl.acm.org/doi/10.1145/3725783.3764393>

RESEARCH-ARTICLE

TARDIS: A GPU-Centric KV Cache Service for Efficient LLM Inference

YIFAN HU, Xiamen University, Xiamen, Fujian, China

SHI QIU, Xiamen University, Xiamen, Fujian, China

JIANQIN YAN, Xiamen University, Xiamen, Fujian, China

HAO CHEN, Xiamen University, Xiamen, Fujian, China

XINTAO WANG, Shanghai Jiao Tong University, Shanghai, China

TANG LU, Xiamen University, Xiamen, Fujian, China

[View all](#)

Open Access Support provided by:

Shanghai Jiao Tong University

Xiamen University

Published: 11 October 2025

[Citation in BibTeX format](#)

APSys '25: 16th ACM SIGOPS Asia-Pacific Workshop on Systems
October 12 - 13, 2025
Seoul, Republic of Korea

Conference Sponsors:
SIGOPS

TARDIS: A GPU-Centric KV Cache Service for Efficient LLM Inference

Yifan Hu*
Xiamen University
Xiamen, China
yifanh@stu.xmu.edu.cn

Shi Qiu*
Xiamen University
Xiamen, China
qiushijxs@stu.xmu.edu.cn

Jianqin Yan
Xiamen University
Xiamen, China
jqyan@stu.xmu.edu.cn

Hao Chen
Xiamen University
Xiamen, China
chenhao2@stu.xmu.edu.cn

Xintao Wang
Shanghai Jiao Tong University
Shanghai, China
xintaowang@sjtu.edu.cn

Lu Tang
Xiamen University
Xiamen, China
tanglu@xmu.edu.cn

Guangtao Xue
Shanghai Jiao Tong University
Shanghai, China
xue-gt@cs.sjtu.edu.cn

Yiming Zhang[†]
XMU & SJTU
Xiamen & Shanghai, China
sdiris@gmail.com

Abstract

Key-value (KV) cache is a crucial optimization for large language model (LLM) serving, particularly in long-context inference scenarios. While existing KV stores suffer from a fundamental mismatch between the CPU-centric KV cache storage and the GPU-accelerated computation: (1) the low-parallelism CPUs cannot satisfy the fine-grained, highly parallel I/O demand, and (2) the synchronization between CPUs and GPUs introduced by layerwise asynchronous KV operations prevents the adoption of low-level optimizations (such as CUDA Graph).

This paper presents *TARDIS*, a GPU-centric KV cache service for efficient long-context LLM inference. Inspired by file-to-memory mapping (mmap) supported by traditional CPU file systems, the key idea of TARDIS is to map KVs directly onto GPU’s high-bandwidth memory (HBM) via modern GPU file systems, so that GPU kernels can access KVs

without CPU intervention thus achieving high-concurrency, fine-grained KV storage/retrieval. At the core of TARDIS is a GPU-driven KV store (called *GStore*), which leverages GeminiFS to allow GPUs to directly access KVs on NVMe SSDs. Based on *GStore*, TARDIS designs an on-GPU scheduler that can adaptively schedule KV requests onto HBM/SSDs, and enables asynchronous, layer-wise token swapping through CUDA Graph, overlapping computation and KV cache access without CPU synchronization overhead. Evaluation reveals that TARDIS significantly outperforms state-of-the-art CPU-centric KV cache designs for LLM inference, boosting serving throughput by up to 20.52% while maintaining performance within 2.04% of an ideal in-memory KV cache.

CCS Concepts

• Information systems → Storage management.

Keywords

GPU Storage, LLM inference, I/O stack, GPU memory

ACM Reference Format:

Yifan Hu, Shi Qiu, Jianqin Yan, Hao Chen, Xintao Wang, Lu Tang, Guangtao Xue, and Yiming Zhang. 2025. TARDIS: A GPU-Centric KV Cache Service for Efficient LLM Inference. In *16th ACM SIGOPS Asia-Pacific Workshop on Systems (APSys ’25)*, October 12–13, 2025, Seoul, Republic of Korea. ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/3725783.3764393>

1 Introduction

The key-value (KV) cache store has emerged as a key component in large language model (LLM) serving. It efficiently manages the massive volumes of KV caches generated during LLM’s autoregressive computation. By restoring historical

*Yifan Hu and Shi Qiu are co-primary authors.

[†]Yiming Zhang is the corresponding author.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

APSys ’25, Seoul, Republic of Korea

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-1572-3/25/10

<https://doi.org/10.1145/3725783.3764393>

KV caches from the KV store when processing identical inputs, LLM frameworks can avoid quadratically increasing computational costs [13]. For example, DeepSeek [6] has demonstrated that KV reuse can reduce inference costs by approximately 74%. Consequently, numerous LLM inference frameworks [14, 19, 20, 23] have integrated external high-capacity KV storage systems [9, 10, 16].

However, existing KV store designs [16, 19, 20] rely on the CPU to orchestrate the KV store and retrieval. This CPU-centric approach results in a fundamental mismatch between the KV cache and GPU-accelerated LLM inference computation, leading to inefficiencies in KV scheduling under real-world workloads. This problem manifests in the following two primary aspects.

First, the limit CPU cores restrict the I/O concurrency. Modern inference frameworks often employ page-table-like abstraction (e.g., PagedAttention [14]) to minimize GPU memory fragmentation, causing KV data to be scattered across non-contiguous physical addresses. In addition, optimizations such as quantization [12], cross-head KV sharing [15], and parallel inference [2], further lower the granularity of data in memory. This leads to a surge of highly concurrent, fine-grained I/O requests from the GPU when storing/retrieving KV data. The low parallelism of CPUs [3, 21, 22] and high software overhead result in underutilization of available bandwidth.

Second, the synchronization between CPUs and GPUs prevents low-level optimizations like CUDA Graph [5, 7, 28]. As a performance optimization tool, CUDA Graph [18] allows frameworks to capture and replay sequences of kernels and memory operations as a single, reusable computational graph, which can effectively reduce the overhead of individual kernel launches and improve execution efficiency. However, CUDA Graph lacks support for dynamic control flow, and memory operations, or synchronization API calls (e.g., `cudaStreamSynchronize()`), which will interrupt the graph capture process. This poses a critical limitation for existing KV cache schemes, as their layer-wise asynchronous KV transmission relies on GPU kernels to delicately synchronize the data.

This paper presents *TARDIS*, a GPU-centric KV cache service for efficient long-context LLM inference. Inspired by file-to-memory mapping (mmap) supported by traditional CPU file systems like ext4, the key idea of *TARDIS* is to map KV data directly onto GPU's HBM via modern GPU file systems like GeminiFS [21], establishing a one-to-one mapping between the KV files and the logical token objects within GPU processes. As a result, GPU kernels can access KV data without CPU intervention thus achieving high-concurrency, fine-grained KV store/retrieval. The core of *TARDIS* is GStore, a GPU-driven KV store that leverages GeminiFS to allow direct GPU access to KVs on NVMe SSDs. Building on GStore,

TARDIS is designed with an on-GPU scheduler that can adaptively schedule KV requests across HBM and SSDs, enabling fully asynchronous, zero-bubble block-level swapping via CUDA Graph. This design allows KV transfers to be seamlessly integrated into the computational graph, and supports efficient overlap between computation and storage access.

We have implemented *TARDIS* as an integrated component of vLLM [14], a SOTA distributed LLM serving framework, to accelerate KV transfer and enable efficient inference for ultra-long context lengths under resource-constrained environments. Experimental results show that *TARDIS* achieves 15.6× to 21× higher bandwidth than SOTA KV Store LM-cache.

2 Background and Motivation

2.1 CPU-Driven KV Stores

In transformer-based LLMs such as GPT-4 [1], inference operates in an autoregressive manner, where the model predicts one token at a time by attending to all previously generated tokens [26]. For an input of length S , the computational complexity of the attention mechanism is $O(S^2)$. The Key-Value (KV) cache addresses this by storing the K and V matrices for historical tokens, which avoids recomputation, reducing the complexity of each decoding step to $O(S)$. Additionally, by reusing the KV cache from previous requests with the same input prompts, LLM inference can avoid redundant computations. This approach further enhances the performance of applications such as multi-turn dialogue systems [32].

However, KV cache also incurs a linear memory overhead proportional to the sequence length. The memory requirement for a single request can be expressed as:

$$M_{KV} = \underbrace{2}_{\text{Key/Value}} \cdot \underbrace{S}_{\text{SeqLen}} \cdot \underbrace{L}_{\text{Layers}} \cdot \underbrace{h}_{\text{Heads}} \cdot \underbrace{d}_{\text{Dimension}} \cdot \underbrace{\text{sizeof}(\text{FP16})}_{2 \text{ Bytes}} \quad (1)$$

Here, S denotes the sequence length, while L , h , and d represent the model's architectural parameters: the number of layers, the number of attention heads, and the head dimension, respectively. To store the massive volume of historical KV, inference frameworks often integrate an external KV store that leverages high-capacity, cost-effective storage solutions that span multiple tiers, such as host memory and high-performance SSDs. However, compared to the terabyte-level TB/s bandwidth provided by GPU HBM, external KV store is limited by PCIe bandwidth (e.g. 64GB/s for PCIe 5.0). As a result, bandwidth utilization has become one of the key factors affecting inference performance.

Existing KV stores mainly use two KV access methods. The first, exemplified by Hcache [10], uses peer-to-peer (P2P) direct transfers. However, it sacrifices memory aggregation, resulting in high-concurrency, small-granularity I/O patterns. The second approach, seen in Mooncake[20] and

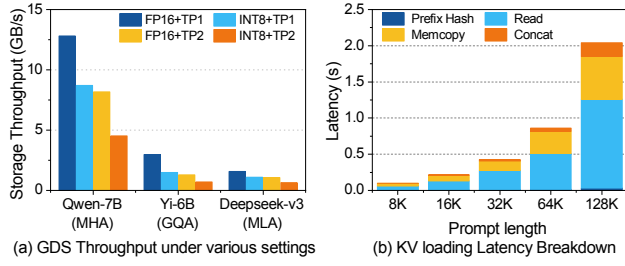


Figure 1: Two CPU-driven KV store performance under Real world workloads.

Method	P2P DMA	Layer-Wise Access	Cuda Graph	Memory Extension
LMcache [16]	×	✓	×	×
Hcache [10]	✓	✓	×	✓

Table 1: Features supported by existing state-of-the-art KV stores.

LMcache[31], uses CPU DRAM as an internal buffer. Here, KV data is first copied from GPU HBM to CPU DRAM, then transformed into parallel I/O requests. This introduces overhead from both data copying and object-level manipulations. Layer-wise asynchronous KV access is another important optimization in KV store design [8, 10], enabling pipelined execution by overlapping computation with fine-grained KV access. Additionally, on-demand layer-wise KV loading and offloading enables HBM to accommodate longer KV caches, thereby supporting extended context lengths.

However, the performance of existing KV stores is suboptimal due to their reliance on the CPU to initiate I/O operations. This dependency leads to two major issues: First, the limited CPU cores and software overhead struggle to handle the high-concurrency, fine-grained I/O requests generated by modern LLM optimizations (e.g., paged memory, parallelism), making it difficult to load long sequences efficiently. Second, synchronization between CPU storage stack and GPU threads prevents the adoption of low-level optimizations like CUDA Graph [27, 29], which require fixed kernel parameters and graph structures, thus limiting performance.

To quantify these performance issues, we evaluated two access ways under real-world LLM inference workloads. For GDS, evaluation varies model quantization, parallelism, and KV sharing strategies. We also measured KV restore latency of LMcache across different prompt lengths and conducted a fine-grained breakdown. Our experiments used a server with 64 CPU cores, an NVIDIA H100 GPU (80GB), and two Solidigm D7-PS1010 NVMe SSDs in RAID 0 (29GB/s read, 20GB/s write peak bandwidth). Fig. 1(a) shows that as the quantization precision decreases and the levels of parallelism

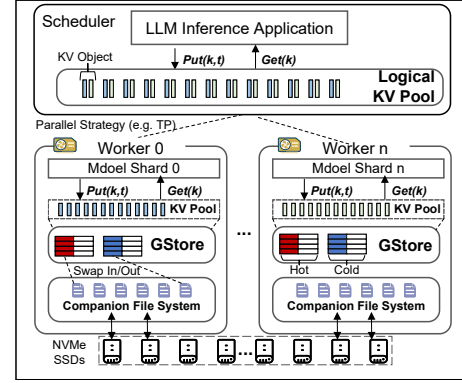


Figure 2: Overview of TARDIS.

and KV cache sharing increase, the effective storage throughput for all three models steadily decreases with throughput reductions of 64.8%, 76.4%, and 58.9%, respectively. Notably, even under optimal conditions, the maximum observed bandwidth never exceeds half of the system’s theoretical peak. For the second access path (Fig. 1(b)), memory copy and concat operations accounted for a substantial 35.2% to 41.0% of total execution time. Finally, Table 1 summarizes the features supported by the state-of-the-art KV store: LMcache and Hcache, which represent the two storage access approaches discussed above. However, none of these existing solutions fully satisfy all KV store access requirements for LLMs.

2.2 GPU-Centric Storage and File Systems

Recent works have explored GPU-centric storage. BaM [22] first introduced novel architectures that leverage a user-level GPU library featuring highly concurrent submission and completion queues situated in GPU memory. Such designs enable GPU threads to access storage devices directly, thereby reducing the need for extensive CPU-GPU synchronization and CPU storage stack overhead. They also significantly improve storage bandwidth utilization for high-concurrency, fine-grained I/O by leveraging GPU parallelism. Building on this, GeminiFS [21] proposes a GPU-centric file system, allowing GPU kernels to directly access host-managed file data. Its storage access interfaces, implemented as GPU kernels, integrate natively into CUDA Graph, enabling data-dependency-based scheduling between compute and storage kernels. This improves execution efficiency and reduces launch overhead, offering a promising path to address the issues of CPU-driven KV stores.

3 TARDIS Design

Inspired by GPU-centric storage, we introduce *TARDIS*, a GPU-centric KV serving for LLM inference framework. The architecture overview of TARDIS is shown in Fig. 2. The core of TARDIS is a GPU-centric KV store called *GStore*, which maps the vast collection of KV files managed by the

GPU-centric file system (GeminiFS) into a pool of coherent high-level KV pairs in GPU HBM. The GStore decentralizes the traditionally centralized KV cache manager and is offloaded to individual GPU workers. This aligns storage access and scheduling decisions with the system’s parallel execution model, enabling fine-grained, scalable, and efficient KV management.

When a new inference request arrives, each GStore autonomously determines the necessary KV entries and performs in-place swap-in or swap-out operations without coordination from a central CPU controller. Across all workers, this distributed architecture exposes a unified logical KV-semantic memory pool to the LLM inference framework, allowing the system to scale seamlessly while maintaining high performance and flexibility. Next, we describe the architecture of GStore and how it achieves efficient KV cache access using Cuda Graph.

3.1 GStore: A GPU-Driven KV Store

To meet the requirements of reusing existing KV pairs and storing newly generated ones during inference, simply mapping KV files into GPU HBM via a GPU-centric file system (like GeminiFS) is insufficient. The system must also support parallel KV management operations, such as concurrent index matching, updates, and block allocation/freeing. Moreover, with the removal of CPU-based scheduling for KV management, a GPU-centric KV store must provide unified memory management between GPU HBM and storage. GStore employs two core components to achieve the above goals: (1) a Distributed KV Manager and (2) Hash-based KV indexing. We describe each in detail below.

3.1.1 Distributed KV manager. The KV manager resides on each GPU worker, managing both GPU HBM and disk storage at a unified granularity. HBM is logically divided into a set of blocks, while the storage is abstracted as *GPU files*. It is worth noting that *GPU files* are an abstraction, and their specific format and content will be discussed in detail in Section 3.1. On the GPU, these are further unified into a shared logical block space: [*mem block*, *file block*]. A uniform block table is introduced to manage the mapping between logical blocks and their physical locations in memory or storage. Each logical block is identified by a *BlockId*, a *Ref* and a *Count*. We use a simple sequential partitioning scheme: if the *BlockId* is less than the maximum number of blocks that can be accommodated in HBM, the block is considered to reside in GPU memory; otherwise, it is located in storage. The *Ref* field indicates the number of requests currently referencing a logical block, while the *Count* field represents the historical reuse count of that block. During system initialization, the framework configures each KV manager according to the aforementioned method and allocates a sufficient number

of *GPU files* through GeminiFS. The size of a logical block is determined jointly by the model configuration and the parallelism strategy used in the framework. As an example, when using tensor parallelism with a degree of 2, the logical block size assigned to each distributed kv manager is divided by two.

3.1.2 Hash-based KV indexing. Hashing is widely used in LLM inference frameworks to accelerate the lookup and management of KV pairs. Frameworks compute the hash of input prompts and maintain a hash table to support $O(1)$ -time KV lookups, allowing for rapid retrieval of previously generated KV pairs. TARDIS also adopts a hash-based indexing mechanism for KV cache management. However, to better align with GPU characteristics, we introduce the following design optimizations.

CPU-GPU Collaborative Hash: Hash computation exhibits sequential dependency: each block’s hash depends on the result of the previous one, making it difficult to parallelize on GPUs. Therefore, we retain hash computation on the CPU, while offloading hash table maintenance tasks to the GPU.

Warp-level Index/Insert: To support efficient block retrieval on the GPU, TARDIS builds a high-performance hash table entirely within GPU memory inspired by GPhash [4]. The hash table is organized into buckets, each sized to match the width of a GPU warp. The GPU warp is the minimum execution unit in CUDA, typically comprising 32 threads. Each bucket entry stores the hash value of a block and its associated *BlockId*. In order to exploit GPU parallelism, GStore organizes hash operations at the warp granularity, allowing for fully parallel index and insert operations.

KV cache restore following the process of KV cache lookup using hashing in TARDIS: (1) Upon receiving a new inference request, the framework partitions the prompt into blocks and computes their hash values on the CPU. (2) These hash values are then transferred to GPU memory via asynchronous memory copy. (3) Once available, the GPU performs parallel retrievals in the hash table to get the location of the corresponding KV blocks. (4) If a *BlockId* is found and resides in HBM, the corresponding GPU thread returns immediately. For blocks stored in storage, the GPU initiates a swap-in operation. (5) If cache miss, the GPU computes the KV states through standard attention operations, allocates new file blocks from the GPU file pool, updates the hash table, and asynchronously persists the resulting KV pairs to storage.

3.2 Scalable GPU File Pool

Existing GPU-centric storage systems, rely on a centralized metadata server to maintain global file state, enforce access control. However, in multi-GPUs settings, this reliance on centralized metadata leads to costly synchronization overhead, as consistency primitives must be used across GPUs to

protect metadata updates, significantly limiting scalability and performance. GStore removes centralized access control and let the framework manage everything. Based on this principle, we design a scalable GPU file pool to address the aforementioned challenges.

3.2.1 GPU-NVMe file mapping. The scalable GPU file pool first decouples the GPU-side file abstraction used by GStore from the underlying NVMe devices through a GPU-NVMe file mapping mechanism. The file pool presents a unified GPU file abstraction to GPU-side applications and manages files distributed across multiple NVMe devices in a coherent manner. GPU files use chunk-level striping to unify multiple NVMe files into a single logical file. This allows the GPU to distribute I/O requests across different storage devices, fully exploiting their parallelism and accelerating KV cache read and write operations.

This decoupled design has flexibility adapting data layouts to match the dynamic scheduling behavior of the inference framework. Even when GPU workers are reassigned due to changes in execution context or resource availability, the system can reconstruct metadata on-the-fly or reconfigure the GPU file abstraction at the granularity of individual files to accommodate new parallelism strategies.

3.2.2 NVMe file formats. TARDIS builds an NVMe File object abstraction on top of the GPU-centric file system. This object maintains essential metadata required for initiating GPU-centric I/O operations, including file size, block mappings, and associated NVMe controller ensuring correct and consistent access to underlying storage devices. TARDIS provides two types of NVMe files to the GPU file pool:

- **KV files:** Used to store the KV generated by the model. After a file is written with KV cache data, its corresponding hash value and its position within the KV matrix are both recorded in the metadata. The format is designed with block-level granularity, aligning with the block management mechanisms of vLLM.
- **Metadata files:** Comprising two subtypes, a meta file that records the usage status of the GPU file pool. A hash file that persists the hash table used for prefix caching.

To ensure crash consistency, each metadata file includes a dirty bitmap to track the modification status of individual file regions or hash table entries. For example, when a KV file is assigned to a new KV and data is written, the corresponding dirty bit is set. The modified data is then asynchronously flushed to persistent storage at a later stage.

3.3 KV Store/Retrieval Based on CUDA Graph

Building on GStore, we provide simple object-level APIs for managing KV Cache, whose interface usage is demonstrated

Listing 1 Pseudo-code for layer-wise scheduling in TARDIS.

```
# init uniform block manager
uniformed_bm = create_uniformed_block_manager(...)
# in Transformer model
for layer_idx in transformers.layers:
    torch.ops._tardis_ops.batch_get(K,V,
                                     uniformed_bm,layer_idx)

    if attn_metadata.common_prefix_len > 0
        cascade_attention(K, V)

    torch.ops._tardis_ops.batch_put(K, V,
                                     layer_idx)
destroy_uniformed_block_manager(device_id)
```

in Listing 1. The KV access pattern is captured by CUDA graphs through analyzing data dependency relationships, which constructs a dataflow graph incorporating both attention kernel and KV store access kernel. This graph is entirely scheduled by the GPU runtime, eliminating explicit CPU-GPU synchronization. Furthermore, KV access kernel are launched asynchronously, enabling optimal computation and KV access overlap. Additionally, our API allows explicit specification of the layer index (`layer_idx`) for KV matrices, enabling fine-grained layer-wise KV access.

4 Evaluation

We conducted a series of testbed experiments to evaluate TARDIS’s performance, focusing on two critical questions:

- (1) How does TARDIS’s bandwidth utilization compare to other approaches?
- (2) How does TARDIS perform in terms of throughput for long context lengths compared to state-of-the-art KV cache services?

Environments. We deployed TARDIS on a 64-core Intel Xeon 6530 server equipped with 512 GB of memory. The server is equipped with an H100 GPU with 80GB HBM and 4× Solidigm D7-PS1010 7.68TB enterprise SSDs [24] which are organized by software raid 0, all located within the same NUMA and interconnected via PCIe 5.0.

Baseline. We first compared GStore against the KV transfer approaches adopted by state-of-the-art KV stores. (a): CudaMemcpyAsync. (b): NVIDIA GDS [17]. For GDS, we use the cufi API provided by NVIDIA to create CPU threads for data transfer. (c): LMcache [16], SOTA KV store that includes DRAM and NVMe SSD.

To evaluate end-to-end performance, we benchmarked TARDIS against two vLLM configurations: one using full recomputation and another equipped with LMCache[16].

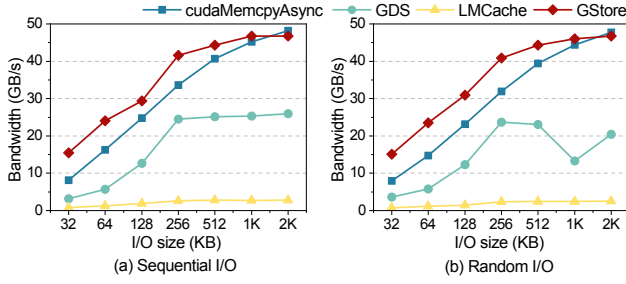


Figure 3: Comparison of read bandwidth of GStore with cudaMemcpyAsync, GDS, LMCache at various I/O size.

4.1 Bandwidth: GStore vs. SOTA Solutions

We first compare GStore with existing KV transfer approaches under real-world conditions by measuring their transmission bandwidth. The evaluation covers KV block sizes ranging from 32KB to 2MB to simulate different quantization precisions and number of tokens in single chunk file. To emulate inference runtime conditions, the benchmark application issues 1,024 parallel I/O requests simultaneously. The results are presented in Fig. 3. As shown in the figure, GStore achieves 15.6× to 21× higher bandwidth utilization than LMCache under both sequential and random I/O. This significant improvement stems from LMCache’s Python-based single-threaded implementation, which struggles with high-concurrency I/O and incurs substantial overhead from additional CPU-GPU memory copies. Furthermore, GStore demonstrates a better bandwidth efficiency of 69.5% to 483.4% compared to GDS, with greater improvements observed at smaller granularities of I/O. These gains originate from two key architectural advantages: (1) direct GPU-initiated I/O eliminates CPU-side software overhead, and (2) GPU threads provide superior I/O parallelism compared to CPU threads (GDS is limited to 128 [30]). In particular, for I/O operations below 1MB, GStore outperforms host memory bandwidth by 3% to 90%. This achievement benefits from both modern NVMe device capabilities and our GPU-driven I/O paradigm, which avoids CPU-GPU communication costs and delivers particularly strong bandwidth gains for fine-grained, high-concurrency workloads.

4.2 End-to-End Workload Performance

We measured the end-to-end system throughput in an offline inference scenario as the number of requests completed per minute to show performance improvement on LLM models: Llama-3.1-8B[11], Yi-6B[33] and Mistral-7B [25]. This evaluation compares three KV access methods: recomputation, LMCache-based retrieval (with data storage varied between DRAM for best-case and SSD for worst-case performance), and TARDIS’s approach. This evaluation uses a total of 100 long-context requests from the LV-Eval[34] workload trace wherein the prompts length per request varies from 46K

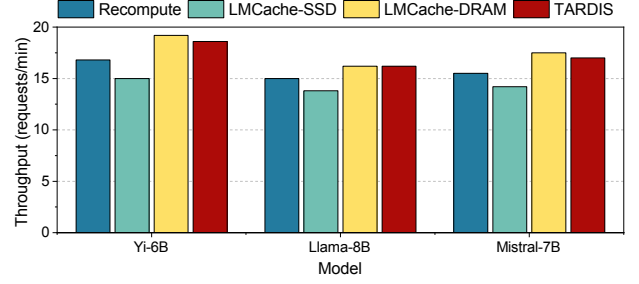


Figure 4: Comparison of throughput of TARDIS with recomputation, LMCache-SSD and LMCache-DRAM

tokens to 56K tokens. And, each request maintained a consistent prefix length of 10,428 tokens, simulating document questioning and answering.

Fig. 4 presents the throughput comparison across TARDIS and baselines. Compared to a fully recomputed approach, both LMCache-DRAM and TARDIS improve 11.79% and 9.51% respectively. However, LMCache-SSD can lead to a throughput degradation of up to approximately 9.13% compared to full recomputation. Despite its design for KV cache reuse, LMCache-SSD underperforms recomputation due to bandwidth utilization constraints. The overhead is associated with frequently accessing or swapping large amounts of small io from the SSD. This bottleneck negates any potential benefits from KV cache reuse, making it less efficient than simply recomputing the entire context in memory for the given workload.

5 Conclusion

This paper presents a GPU-centric KV store (TARDIS) for LLM inference framework. TARDIS map KVs directly onto GPU’s HBM via modern GPU file systems and implement a CPU-GPU collaborative hashing to enables high concurrent KV index. This allows GPU kernels can access KVs without CPU intervention thus achieving high-concurrency, fine-grained KV store/retrieval. Additionally, TARDIS designs an on-GPU scheduler that can adaptively schedule KV requests onto HBM/SSDs, and enables asynchronous, layer-wise token swapping through CUDA Graph, overlapping computation and KV cache access without CPU synchronization overhead.

Acknowledgment

We thank our shepherd, Prof. Sudarsun Kannan, and the anonymous reviewers for their valuable suggestions. This work is supported by the National Key Research and Development Program of China (grant no. 2022YFB4500302) and the National Natural Science Foundation of China (grant no. 62441220). Yiming Zhang is the corresponding author.

References

- [1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altmenschmidt, Sam Altman, Shyamal Anadkat, et al. 2023. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774* (2023).
- [2] Amey Agrawal, Junda Chen, Íñigo Goiri, Ramachandran Ramjee, Chaojie Zhang, Alexey Tumanov, and Esha Choukse. 2024. Mnemosyne: Parallelization strategies for efficiently serving multi-million context length llm inference requests without approximations. *arXiv preprint arXiv:2409.17264* (2024).
- [3] Chia-Hao Chang, Jihoon Han, Anand Sivasubramaniam, Vikram Sharma Mailthody, Zaid Qureshi, and Wen-Mei Hwu. 2024. GMT: GPU Orchestrated Memory Tiering for the Big Data Era. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. 464–478.
- [4] Menglei Chen, Yu Hua, Zhangyu Chen, Ming Zhang, and Gen Dong. 2025. {GPHash}: An Efficient Hash Index for {GPU} with {Byte-Granularity} Persistent Memory. In *23rd USENIX Conference on File and Storage Technologies (FAST 25)*. 203–220.
- [5] datacrunch.io. 2025. DeepSeek-V3 + SGLang: Inference Optimization. <https://datacrunch.io/blog/deepseek-v3-sglang-inference-optimization>.
- [6] Deepseek. 2025. Models and Pricing. https://api-docs.deepseek.com/quick_start/pricing/.
- [7] Jonah Ekelund, Stefano Markidis, and Ivy Peng. 2025. Boosting Performance of Iterative Applications on GPUs: Kernel Batching with CUDA Graphs. *arXiv preprint arXiv:2501.09398* (2025).
- [8] Bin Gao, Zhuomin He, Puru Sharma, Qingxuan Kang, Djordje Jevdjic, Junbo Deng, Xingkun Yang, Zhou Yu, and Pengfei Zuo. 2024. Attention-Store: Cost-effective Attention Reuse across Multi-turn Conversations in Large Language Model Serving. *arXiv preprint arXiv:2403.19708* (2024).
- [9] Bin Gao, Zhuomin He, Puru Sharma, Qingxuan Kang, Djordje Jevdjic, Junbo Deng, Xingkun Yang, Zhou Yu, and Pengfei Zuo. 2024. {Cost-Efficient} large language model serving for multi-turn conversations with {CachedAttention}. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)*. 111–126.
- [10] Shiwei Gao, Youmin Chen, and Jiwu Shu. 2025. Fast state restoration in llm serving with hcache. In *Proceedings of the Twentieth European Conference on Computer Systems*. 128–143.
- [11] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. 2024. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783* (2024).
- [12] Coleman Hooper, Sehoon Kim, Hiva Mohammadzadeh, Michael W Mahoney, Sophia Shao, Kurt Keutzer, and Amir Gholami. 2024. Kvquant: Towards 10 million context length llm inference with kv cache quantization. *Advances in Neural Information Processing Systems* 37 (2024), 1270–1303.
- [13] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*. 611–626.
- [14] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*.
- [15] Aixiu Liu, Bei Feng, Bin Wang, Bingxuan Wang, Bo Liu, Chenggang Zhao, Chengqi Deng, Chong Ruan, Damai Dai, Daya Guo, et al. 2024. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. *arXiv preprint arXiv:2405.04434* (2024).
- [16] Yuhao Liu, Hanchen Li, Yihua Cheng, Siddhant Ray, Yuyang Huang, Qizheng Zhang, Kuntai Du, Jiayi Yao, Shan Lu, Ganesh Ananthanarayanan, et al. 2024. Cachegen: Kv cache compression and streaming for fast large language model serving. In *Proceedings of the ACM SIGCOMM 2024 Conference*. 38–56.
- [17] Nvidia. 2024. NVIDIA GPUDirect Storage. <https://docs.nvidia.com/gpudirect-storage/index.html>.
- [18] NVIDIA. 2025. Creating a Graph Using Graph APIs. <https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html>.
- [19] NVIDIA. 2025. NVIDIA Dynamo. <https://github.com/ai-dynamo/dynamo/>.
- [20] Ruoyu Qin, Zheming Li, Weiran He, Mingxing Zhang, Yongwei Wu, Weimin Zheng, and Xinran Xu. 2024. Mooncake: Kimi’s KVCache-centric Architecture for LLM Serving. *arXiv preprint arXiv:2407.00079* (2024).
- [21] Shi Qiu, Weinan Liu, Yifan Hu, Jianqin Yan, Zhirong Shen, Xin Yao, Renhai Chen, Gong Zhang, and Yiming Zhang. 2025. GeminiFS: A Companion File System for GPUs. In *23rd USENIX Conference on File and Storage Technologies (FAST 25)*. USENIX Association, Santa Clara, CA, 221–236. <https://www.usenix.org/conference/fast25/presentation/qiu>
- [22] Zaid Qureshi, Vikram Sharma Mailthody, Isaac Gelado, Seungwon Min, Amna Masood, Jeongmin Park, Jinjun Xiong, Chris J Newburn, Dmitri Vainbrand, I-Hsin Chung, et al. 2023. GPU-initiated on-demand high-throughput storage access in the BaM system architecture. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*. 325–339.
- [23] SGLang. 2024. SGLang. <https://github.com/sgl-project/sglang?tab=readme-ov-file>.
- [24] solidigm. 2024. Solidigm D7-PS1010. <https://www.solidigm.com/products/data-center/d7/ps1010.html>.
- [25] Hiren Thakkar and A Manimaran. 2023. Comprehensive Examination of Instruction-Based Language Models: A Comparative Analysis of Mistral-7B and Llama-2-7B. In *2023 International Conference on Emerging Research in Computational Science (ICERCS)*. IEEE, 1–6.
- [26] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).
- [27] vllm. 2025. vLLM V1 - Default max CUDA graph size. <https://discuss.vllm.ai/t/vllm-v1-default-max-cuda-graph-size/357/>.
- [28] vllm. 2025. vLLM V1: A Major Upgrade to vLLM’s Core Architecture. <https://blog.vllm.ai/2025/01/27/v1-alpha-release.html>.
- [29] vllm. 2025. vLLM’s torch.compile integration. https://github.com/vllm-project/vllm/blob/main/docs/source/design/v1/torch_compile.md/.
- [30] Devavret Makkar Vukasin Milovanovic and Gregory Kimball. 2022. Boosting Data Ingest Throughput with GPUDirect Storage and RAPIDS cuDF. <https://developer.nvidia.com/zh-cn/blog/boosting-data-ingest-throughput-with-gpudirect-storage-and-rapids-cudf/>.
- [31] Jiayi Yao, Hanchen Li, Yuhao Liu, Siddhant Ray, Yihua Cheng, Qizheng Zhang, Kuntai Du, Shan Lu, and Junchen Jiang. 2025. CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion. In *Proceedings of the Twentieth European Conference on Computer Systems*. 94–109.
- [32] Zihao Yi, Jiarui Ouyang, Yuwen Liu, Tianhao Liao, Zhe Xu, and Ying Shen. 2024. A survey on recent advances in llm-based multi-turn dialogue systems. *arXiv preprint arXiv:2402.18013* (2024).
- [33] Alex Young, Bei Chen, Chao Li, Chengen Huang, Ge Zhang, Guanwei Zhang, Guoyin Wang, Heng Li, Jiangcheng Zhu, Jianqun Chen,

- et al. 2024. Yi: Open foundation models by 01. ai. *arXiv preprint arXiv:2403.04652* (2024).
- [34] Tao Yuan, Xuefei Ning, Dong Zhou, Zhijie Yang, Shiyao Li, Minghui Zhuang, Zheyue Tan, Zhuyu Yao, Dahua Lin, Boxun Li, Guohao Dai, Shengen Yan, and Yu Wang. 2024. LV-Eval: A Balanced Long-Context Benchmark with 5 Length Levels Up to 256K. arXiv:2402.05136 [cs.CL]